

Bài tập chương VI

Câu 1:

a. “**Tương tranh**” là gì? Hãy định nghĩa khái niệm tương tranh theo Edsger Dijkstra và nêu hai hiện tượng phổ biến (mất cập nhật, đọc không nhất quán) kèm theo mô tả ngắn về nguyên nhân xảy ra của mỗi hiện tượng.

- Tương tranh là hiện tượng tại một thời điểm có nhiều yêu cầu sử dụng tài nguyên dùng chung
- Định nghĩa tương tranh theo Edsger Dijkstra: "Tương tranh xảy ra khi nhiều hơn một luồng thực thi có thể chạy đồng thời"

→ Nhiều tiến trình truy nhập đồng thời đến các tài nguyên dùng chung như bộ nhớ, kênh truyền thông, các tập tin trên ổ đĩa... thì có thể dẫn tới hiện tượng xung đột và nó là một trong những nguyên nhân cơ bản làm suy giảm hiệu năng hệ thống, thậm chí có thể làm treo hệ thống.

- “Mất cập nhật” là hiện tượng khi hai hoặc nhiều tiến trình cùng đọc và cập nhật một dữ liệu chung nhưng chúng cùng đọc dữ liệu trước khi cập nhật khiến dữ liệu bị mất cập nhật hoặc kết quả trở nên sai lệch

→ Nguyên nhân do các giao dịch cùng **đọc - ghi** lên dữ liệu **mà không có kiểm soát đồng thời (concurrency control)** như khoá (lock), phiên bản (version), hoặc rollback khi xung đột.

- “Đọc không nhất quán” là hiện tượng người dùng đọc dữ liệu, thì dữ liệu đó bị thay đổi bởi một giao dịch khác trước khi quá trình đọc hoàn tất, dẫn đến kết quả đọc bị sai lệch, không đồng nhất hoặc không phản ánh đúng trạng thái dữ liệu tại một thời điểm nhất định.

→ Nguyên nhân do giao dịch đang đọc dữ liệu, nhưng trong lúc đó, một giao dịch khác **cập nhật hoặc xoá** phần dữ liệu đang được đọc. Do đó, dữ liệu **trong cùng một lượt đọc** có thể không thống nhất, gây ra lỗi logic.

b. Dựa trên ví dụ chuyển tiền giữa các tài khoản A, B, C, hãy giải thích tại sao trình tự thực thi không đồng bộ của các lệnh **GetBalance/SetBalance/Withdraw** dẫn đến hiện tượng mất cập nhật và đọc không nhất quán.

Giả sử ba tài khoản A, B và C đều thuộc về một ngân hàng, nghĩa là các tài khoản này đều được lưu trữ trong một cơ sở dữ liệu, số dư tài khoản của chúng có giá trị lần lượt là 100\$, 200\$ và 300\$. Hai tài khoản A và C cùng chuyển cho tài khoản B số tiền bằng 10% số dư tài khoản hiện có của B.

Trình tự thực hiện gây mất cập nhật:

1. Giao dịch 1 (từ A): đọc số dư B = 200 : float Balance= B.GetBalance();
Rồi tính tiền cần chuyển : float Trans= Balance/0.1;
 2. Giao dịch 2 (từ C): cũng đọc số dư B = 200 : float Balance= B.GetBalance();
Rồi tính tiền cần chuyển : float Trans= Balance*0.1;
- Do thao tác cập nhật B sau khi chuyển từ A chưa được thực hiện nên khi C thực hiện vẫn dùng dữ liệu B trước khi cập nhật
3. Giao dịch 1: tính $200 + 20 = 220$, ghi lại B = 220
 4. Giao dịch 2: tính $200 + 20 = 220$, cũng ghi lại B = 220 → **ghi đè lên kết quả trước**
 - **Lẽ ra B phải là 240\$, nhưng chỉ còn 220\$, và một giao dịch bị mất cập nhật → mất cập nhật đã xảy ra.**

Trình tự gây đọc không đồng bộ:

1. Khi A chuyển cho B 10\$: A.Withdraw(10);
2. float Total=A.GetBalance();
Total= Total+ B.GetBalance();
→ Tiền được tính tổng nhưng chưa cập nhật lại B, nên tổng là $90 + 200 = 290\$$
3. Sau đó, tài khoản B được cập nhật B.Deposit(10);
4. Do B đã được cập nhật nên khi nhân viên check lần nữa sẽ thấy kết quả là 300\$ (= 210 + 90)
 - Dữ liệu của B tại hai thời điểm là khác nhau (trước và sau cập nhật) nên khi đọc dữ liệu này sẽ dẫn đến hiện tượng “đọc không đồng bộ”.

c. Cho đoạn mã MoneyTransfer không dùng giao tác, hãy chỉ ra ít nhất hai điểm cần sửa đổi (hoặc bổ sung cơ chế khóa, giao tác) để đảm bảo tính toàn vẹn dữ liệu, đồng thời minh họa bằng pseudocode (hoặc giải thuật) cho phân xử lý giao tác.

Đoạn mã MoneyTransfer không sử dụng giao tác đang tiềm ẩn **hai lỗi nghiêm trọng** là:

1. **Mất cập nhật (Lost Update)** – Khi hai giao dịch đồng thời đọc cùng một giá trị và cập nhật lại mà không có sự đồng bộ, làm ghi đè kết quả lẫn nhau.
2. **Đọc không nhất quán (Inconsistent Read)** – Khi một giao dịch đọc dữ liệu đang bị thay đổi bởi một giao dịch khác, dẫn đến kết quả sai lệch.

Hai điểm cần sửa đổi

1. Bổ sung khối giao tác (Transaction Block)

Gộp toàn bộ các lệnh trong **MoneyTransfer** vào một giao tác đảm bảo tính **nguyên tử** và **cô lập**. Điều này ngăn cản việc xen kẽ giữa các giao tác khác nhau.

2. Áp dụng cơ chế khóa (Concurrency Control)

Sử dụng **khóa ghi (write-lock)** trên cả hai tài khoản để đảm bảo **tuần tự hóa** khi truy cập và cập nhật tài khoản.

Pseudocode minh họa xử lý có giao tác và khoá nghiêm ngặt

pseudo

CopyEdit

```
MoneyTransfer(Accounts X, Accounts Y, float Z)
{
    BEGIN_TRANSACTION; // Mở giao tác

    LOCK_WRITE(Y); // Thiết lập khóa ghi cho tài khoản Y
    LOCK_WRITE(X); // Thiết lập khóa ghi cho tài khoản X

    float BalanceY = Y.GetBalance(); // Đọc số dư tài khoản Y
    float TransAmount = BalanceY * Z; // Tính tiền cần chuyển

    Y.SetBalance(BalanceY + TransAmount); // Cộng tiền vào tài khoản Y
    X.Withdraw(TransAmount); // Trừ tiền tài khoản X

    COMMIT; // Cam kết giao tác: thay đổi có hiệu lực vĩnh viễn
    UNLOCK(Y);
    UNLOCK(X);
}
```

Trong mô hình này, việc đọc – tính – ghi của từng giao tác đều nằm **trong một khối giao tác tuần tự**, đồng thời giữ **khóa ghi nghiêm ngặt** cho tới khi **COMMIT** đảm bảo tính nhất quán.

d. Phân tích chi tiết một kịch bản cụ thể (nêu trình tự 8 câu lệnh) trong ví dụ chuyển tiền đồng thời giữa A và C với B, chỉ ra nguyên nhân dẫn đến

sai lệch kết quả (số dư B = 220\$ thay vì ≥ 240 \$) và chỉ rõ ở đâu còn sơ hở nếu chỉ gom 3 câu lệnh đầu vào giao tác.

Tình huống lỗi: thực hiện xen kẽ không kiểm soát

ST T	Câu lệnh	Diễn giải	Trạng thái tạm thời (B, A, C)
1	<code>float Balance = B.GetBalance();</code>	A đọc số dư của B: 200\$	
2	<code>float Trans = Balance * 0.1;</code>	A tính Trans = 20\$	
3	<code>float Balance = B.GetBalance();</code>	C đọc số dư của B: vẫn là 200\$	
4	<code>float Trans = Balance * 0.1;</code>	C tính Trans = 20\$	
5	<code>B.SetBalance(Balance + Trans);</code>	A cập nhật: $200 + 20 = 220$ \$	B = 220\$
6	<code>A.Withdraw(Trans);</code>	A bị trừ: $100 - 20 = 80$ \$	A = 80\$
7	<code>B.SetBalance(Balance + Trans);</code>	C cũng cập nhật: $200 + 20 = 220$ \$ (lỗi!)	B bị ghi đè lại thành 220\$
8	<code>C.Withdraw(Trans);</code>	C bị trừ: $300 - 20 = 280$ \$	C = 280\$

Kết quả sai lệch

- Tổng số tiền trừ từ A và C: $20\$ + 20\$ = 40\$$
- Số dư của B chỉ tăng lên từ 200\$ → 220\$, nghĩa là chỉ tăng 20\$, mất 20\$
- ⇒ Dữ liệu bị mất cập nhật, không phản ánh đúng số tiền đã chuyển

Nguyên nhân chính: Mất cập nhật (Lost Update)

Câu lệnh `B.SetBalance(...)` của C đã ghi đè kết quả mà A vừa cập nhật, bởi vì cả hai cùng tính dựa trên **Balance = 200**, trong khi A đã làm thay đổi giá trị này trước đó nhưng C không biết do thiếu đồng bộ.

Sơ hở khi chỉ gom 3 câu lệnh đầu vào giao tác

Nếu chỉ gom:

```
float Balance = B.GetBalance();  
float Trans = Balance * 0.1;  
B.SetBalance(Balance + Trans);
```

vào **giao tác** mà để câu `Withdraw()` nằm ngoài, ta vẫn gặp **lỗi** sau:

- Giao tác B đã **cộng tiền** vào tài khoản B và **COMMIT** thành công
- Nhưng nếu thao tác `A.Withdraw(Trans)` bị **lỗi** (ví dụ: **A không đủ tiền**) thì tiền vẫn đã cộng vào B
- $\Rightarrow$ **Lỗi nghiệp vụ**: tiền chưa thực sự được trừ từ người gửi mà người nhận đã được cộng

e. Thiết kế một kiến trúc hoặc cơ chế điều khiển tương tranh cho hệ thống phân tán lưu trữ giao dịch ngân hàng, sao cho vừa đảm bảo:

- Không xảy ra mất cập nhật, đọc không nhất quán;
- Giảm thiểu khả năng khóa chết và “đói tài nguyên”;
- Vận hành hiệu quả trên môi trường nhiều nút, nhiều bản sao dữ liệu.

Hãy trình bày mô hình dữ liệu, cơ chế khóa/giao tác hoặc sử dụng kỹ thuật không khóa (lock-free), và đánh giá ngắn ưu – nhược điểm của giải pháp bạn đưa ra.

Thiết kế kiến trúc điều khiển tương tranh cho hệ thống phân tán lưu trữ giao dịch ngân hàng

1. Mô hình dữ liệu

Hệ thống bao gồm nhiều nút lưu trữ phân tán, mỗi nút chứa một phần dữ liệu ngân hàng (phân vùng theo tài khoản khách hàng hoặc chi nhánh). Các nút có thể **nhân bản dữ liệu quan trọng** (ví dụ **tài khoản nhận tiền**) để đảm bảo tính sẵn sàng cao.

- Dữ liệu được chia thành **các vùng truy cập phổ biến**:
 - **Vùng đọc nhiều** (ví dụ: truy vấn số dư, lịch sử giao dịch)
 - **Vùng ghi thường xuyên** (ví dụ: cập nhật số dư khi chuyển khoản)

Mỗi vùng sẽ áp dụng **cơ chế điều khiển tương tranh phù hợp**.

2. Cơ chế điều khiển tương tranh đề xuất: Hybrid (Kết hợp)

Sử dụng **kết hợp ba phương pháp**: Khóa hai pha (2PL), Điều khiển lạc quan (OCC), và Nhãn thời gian (TSO) tùy vào đặc điểm của phân vùng và loại giao dịch:

Loại dữ liệu/giao dịch	Phương pháp áp dụng	Lý do
Truy vấn số dư, đọc lịch sử	Không khóa / OCC	Đọc không xung đột $\Rightarrow$ tăng tính song song
Giao dịch chuyển tiền, ghi số dư	Khóa hai pha nghiêm ngặt (Strict 2PL)	Đảm bảo tính nhất quán và tương đương tuần tự
Cập nhật đồng thời tài khoản hot	Dựa trên nhãn thời gian (TSO)	Tránh deadlock, tối ưu hiệu năng

3. Cơ chế khóa và xử lý giao dịch

3.1. Khóa hai pha nghiêm ngặt (Strict 2PL) cho giao dịch ghi:

- Khi giao dịch **đọc số dư**: thiết lập **khóa đọc**
- Khi giao dịch **ghi số dư**: nâng lên **khóa ghi**
- **Chỉ giải phóng khóa khi giao dịch commit**, giúp đảm bảo tính tuần tự.

Giải quyết deadlock:

- Dùng **đồ thị chờ đợi** và **giám sát timeout**
- Hủy giao dịch chờ quá lâu, khôi phục trạng thái nhất quán bằng log.

3.2. Điều khiển lạc quan (OCC) cho truy vấn đọc:

- Giao dịch đọc thao tác trên **bản sao tạm thời**
- Khi commit sẽ kiểm tra xem **có xung đột với các giao dịch đã commit không**
 - Nếu không: cập nhật thành công
 - Nếu có: hủy giao dịch, client sẽ thực hiện lại

Ưu điểm: giảm tải hệ thống, tăng tốc độ truy vấn

Nhược điểm: giao dịch có thể bị rollback khi có xung đột

3.3. Nhãn thời gian (TSO) cho vùng ghi đồng thời:

- Mỗi giao dịch gán nhãn thời gian duy nhất
- Các thao tác ghi/đọc bị từ chối nếu vi phạm thứ tự nhãn
- Cho phép ghi đồng thời mà **tránh được deadlock**

4. Đánh giá giải pháp

Tiêu chí	Ưu điểm	Nhược điểm
Đảm bảo nhất quán	Strict 2PL đảm bảo tương đương tuần tự	Có thể gây khóa chết nếu không có xử lý tốt
Tăng hiệu năng đọc	OCC cho phép nhiều giao dịch đọc đồng thời	Có thể rollback nếu bị ghi xung đột trong quá trình commit
Tránh deadlock	TSO tránh tình trạng chờ vòng bằng thứ tự nhãn thời gian	Phức tạp trong cài đặt và phải theo dõi thứ tự chặt chẽ
Tối ưu hóa theo từng phân vùng	Phân tách theo đặc điểm dữ liệu giúp tối ưu hiệu suất tổng thể	Cần thiết kế và giám sát kỹ càng để đảm bảo phối hợp hiệu quả

Câu 2:

a. Trình bày định nghĩa “giao tác” trong hệ quản trị cơ sở dữ liệu phân tán và nêu rõ bốn tính chất ACID của giao tác

- Giao tác gồm một hoặc nhiều câu lệnh truy nhập dữ liệu và chúng được thực hiện như một đơn vị thống nhất, nghĩa là không thể thêm hay bớt bất kỳ câu lệnh nào. Giao tác được chia thành giao tác phẳng, giao tác lồng ghép và giao tác phân tán.
- Giao tác có thể gồm nhiều câu lệnh đọc/ghi dữ liệu và phải đảm bảo bốn tính chất: nguyên tử, nhất quán, cô lập và bền vững.
 - Tính chất nguyên tử nghĩa là đối với thế giới bên ngoài thì giao tác không thể chia nhỏ hơn được, các lệnh trong giao tác đều được thực hiện hoặc không có lệnh nào được thực hiện, nếu giao tác bị hủy giữa chừng thì kết quả của các lệnh đã thực hiện sẽ bị hủy bỏ.
 - Tính chất nhất quán nghĩa là luôn phải đảm bảo tính toàn vẹn dữ liệu, giao tác không vi phạm tính chất bất biến của hệ thống.

- Tính chất cô lập nghĩa là mỗi giao tác sẽ hoạt động độc lập và không làm ảnh hưởng đến các giao tác khác.
- Tính chất bền vững nghĩa là khi giao tác đã cam kết thì các thay đổi đối với nó không phải là tạm thời mà là những thay đổi bền vững, nếu không cam kết thì sẽ tự động phục hồi về giá trị cũ, coi như chưa từng thực hiện các lệnh trong giao tác.

b. Giải thích sự khác nhau cơ bản giữa giao tác phẳng, giao tác lồng ghép và giao tác phân tán. Cho ví dụ minh họa đơn giản cho từng loại.

Loại giao tác	Sự khác nhau cơ bản
Giao tác phẳng	Giao tác đơn tuyến, thực hiện toàn bộ hoặc không gì cả, không chia nhỏ được.
Giao tác lồng ghép	Giao tác lớn gồm nhiều giao tác con, có thể xử lý và phục hồi cục bộ, nhưng phụ thuộc vào giao tác cha.
Giao tác phân tán	Giao tác thực hiện trên nhiều hệ thống khác nhau, cần phối hợp đồng bộ giữa các hệ thống.

Ví dụ: Giao tác phẳng

Một giao tác chuyển tiền:

BEGIN TRANSACTION

- Trừ 500.000 đồng khỏi tài khoản A
- Cộng 500.000 đồng vào tài khoản B

COMMIT

Nếu một trong hai bước thất bại, giao tác bị **ROLLBACK** và không có thay đổi nào được ghi lại.

Ví dụ: Giao tác lồng ghép

Giao tác "Đặt hàng" bao gồm các giao tác con:

- T1: Kiểm tra tồn kho
- T2: Trừ hàng trong kho

- T3: Tạo đơn hàng

T (Giao tác cha)

- |— T1: Kiểm tra tồn kho
- |— T2: Trừ hàng
- |— T3: Tạo đơn hàng

Nếu T2 thất bại, T cha sẽ hủy toàn bộ và phục hồi dữ liệu của T1, T2, T3.

Ví dụ: Giao tác phân tán

Một hệ thống thương mại điện tử:

- Giao dịch trừ tiền (trên ngân hàng A – Server 1).
- Giao dịch tạo đơn hàng (trên hệ thống đặt hàng – Server 2).

Trình tự:

1. Server 1: Trừ tiền => OK
2. Server 2: Tạo đơn hàng => OK
3. **Cả hai đều cam kết** => Giao tác thành công.

Nếu một trong hai thất bại => **Tất cả rollback**.

c. Viết lại bằng cách đóng gói thành một giao tác phẳng, kèm theo các lệnh mở giao tác, COMMIT và ROLLBACK.

```
FlatTransaction_UpdateAccount(A, delta) {
    BEGIN_TRANSACTION;      // Mở giao tác

    try {
        balance = Read(A);
        Write(A, balance + delta);

        COMMIT;             // Cam kết giao tác nếu không lỗi
    } catch (Exception e) {
        ROLLBACK;           // Hủy giao tác nếu có lỗi xảy ra
    }
}
```

d. Phân tích ưu nhược điểm của giao tác lồng ghép so với giao tác phẳng trong kịch bản: một giao dịch lớn T bao gồm hai phần độc lập T1

và T2 cùng truy cập một số tài nguyên chung. Nêu rõ cách đảm bảo tính cô lập và nguyên tử ở cả cấp cha và cấp con.

Kịch bản phân tích:

Một giao dịch lớn **T** gồm hai phần độc lập **T1** và **T2**, cùng truy cập một số tài nguyên chung (ví dụ: cùng đọc/ghi vào một bảng dữ liệu), cần đảm bảo:

- **Tính nguyên tử:** hoặc cả T1 và T2 thành công, hoặc không phần nào được giữ lại.
- **Tính cô lập:** T1 và T2 không gây ảnh hưởng sai lệch tới nhau hoặc tới các giao tác khác.

1. Giao tác phẳng

Ưu điểm:

- **Đơn giản** trong triển khai và kiểm soát: chỉ có một mức giao tác, dễ rollback/commit.
- **Tính nguyên tử và cô lập** dễ đảm bảo ở cấp cao nhất, vì toàn bộ T1 và T2 nằm gộp trong một giao tác đơn.

Nhược điểm:

- **Thiếu linh hoạt:** không thể commit hoặc rollback riêng T1 hoặc T2.
- **Khả năng tái sử dụng kém:** không thể xử lý T1 và T2 như những đơn vị tái sử dụng được trong các ngữ cảnh khác.
- Nếu **T1 thất bại**, toàn bộ **T**, kể cả **T2**, cũng bị rollback => gây lãng phí nếu **T2** đã thực hiện thành công.
- Trong trường hợp **T1** và **T2** cùng truy cập tài nguyên chung, cần khóa toàn bộ giao tác lớn **T**, gây **giảm tính song song**.

2. Giao tác lồng ghép

Ưu điểm:

- **Chia nhỏ logic:** Giao tác lớn **T** có thể chia thành **T1** và **T2**, mỗi phần có thể xử lý độc lập và tái sử dụng.
- Cho phép **xử lý song song**: **T1** và **T2** (nếu không xung đột tài nguyên) có thể chạy đồng thời.
- **Cách ly và kiểm soát lỗi tốt hơn:** nếu **T1** thất bại, có thể rollback **T1** mà không ảnh hưởng đến **T2** tức thì, quyết định cuối cùng do **T** xử lý.
- Hệ thống có thể **tuần tự hóa truy cập tài nguyên chung**, giữ đúng tính nhất quán.

Nhược điểm:

- **Quản lý phức tạp hơn:** phải đảm bảo rằng giao tác cha **T** không cam kết cho đến khi tất cả các giao tác con hoàn tất.
- Nếu **T1** và **T2** cùng truy cập tài nguyên chung → cần thiết kế **cơ chế khóa cấp con và đồng bộ** để tránh xung đột (ví dụ: thứ tự truy cập, dùng lock).
- Nếu **T** bị rollback → tất cả **T1** và **T2** đều bị hủy, kể cả khi một trong hai đã commit → gây **lãng phí tài nguyên**.

Đảm bảo tính nguyên tử và cô lập:

Tính chất	Giao tác phẳng	Giao tác lồng ghép
Nguyên tử	Đảm bảo ở mức giao tác toàn cục (T)	- Đảm bảo ở từng cấp - Giao tác con nguyên tử với cha - Cha quyết định cuối cùng
Cô lập	Cả T1 và T2 được thực hiện tuần tự hoặc có khóa	Các giao tác con cùng cấp có thể chạy song song, truy cập chung phải được tuần tự hóa

e. Thiết kế một cơ chế giao tác phân tán cho hệ thống ngân hàng phân tán nhiều nút, đảm bảo tính ACID. Trong đó:

- 1. Mô tả quy trình hai pha (Two-Phase Commit) hoặc ba pha (Three-Phase Commit).**
- 2. Đánh giá khả năng xảy ra “khóa chết phân tán” và đề xuất biện pháp giảm thiểu.**

3. Trình bày cách xử lý khi một nút tham gia cam kết bị lỗi giữa chừng.

Bối cảnh:

Hệ thống ngân hàng có nhiều nút (server), mỗi nút lưu dữ liệu của một số tài khoản. Một giao dịch chuyển tiền từ tài khoản A (ở nút 1) sang tài khoản B (ở nút 2) cần:

- Cùng trừ tiền từ A và cộng vào B
- Đảm bảo **toàn vẹn (Consistency)** và **nguyên tử (Atomicity)** dù các nút hoạt động độc lập

1. Mô tả quy trình Two-Phase Commit (2PC)

Hai pha:

Pha 1: Chuẩn bị (Prepare Phase)

- **Coordinator (bộ điều phối)** gửi lệnh "**prepare**" đến tất cả các nút tham gia (Participants).
- Mỗi nút:
 - Ghi log "chuẩn bị cam kết"
 - Khóa tài nguyên liên quan (đảm bảo tính **cô lập**)
 - Trả về "**OK**" nếu sẵn sàng hoặc "**ABORT**" nếu gặp lỗi

Pha 2: Cam kết (Commit Phase)

- Nếu tất cả nút trả "**OK**":
 - Coordinator gửi "**COMMIT**" cho tất cả các nút
 - Mỗi nút thực hiện cập nhật thật sự → ghi log "**commit**" và **mở khóa**
- Nếu bất kỳ nút nào trả "**ABORT**":
 - Coordinator gửi "**ROLLBACK**"

- Mỗi nút khôi phục về trạng thái ban đầu

Tính chất ACID đảm bảo:

- **Nguyên tử**: hoặc tất cả commit, hoặc rollback toàn bộ
- **Nhất quán**: mọi cập nhật đều hợp lệ
- **Cô lập**: khóa trong pha chuẩn bị
- **Bền vững**: nhờ ghi log trước khi thực hiện

2. Đánh giá nguy cơ “khóa chết phân tán” & biện pháp giảm thiểu

Khả năng xảy ra:

- Trong pha chuẩn bị, các nút **khóa tài nguyên** nhưng không commit được (ví dụ: coordinator bị treo).
- Dẫn đến trạng thái "chờ mãi mãi", không thể tiếp tục giao dịch khác → “**khóa chết phân tán**” (distributed deadlock).

Biện pháp giảm thiểu:

1. Thay 2PC bằng 3PC (Three-Phase Commit):

- Thêm một bước “**canCommit**” trước khi gửi “prepare” giúp hệ thống tránh trạng thái không thể quyết định khi coordinator lỗi.
- Giảm tình trạng "mắc kẹt"

2. Đặt timeout cho các bước:

- Nếu nút chờ quá lâu không thấy phản hồi từ coordinator → tự động rollback

3. Dùng giao thức cấp cao như Paxos hoặc Raft:

- Hỗ trợ phục hồi từ lỗi tốt hơn, tránh treo hệ thống

4. Thiết kế các giao tác nhỏ hơn, ngắn hơn:

- Giảm thời gian giữ khóa, từ đó giảm nguy cơ deadlock

3. Xử lý khi một nút bị lỗi giữa cam kết

Tình huống:

- Nút 2 bị lỗi sau khi gửi "OK" nhưng **chưa thực hiện COMMIT**
- Coordinator không nhận được phản hồi => phải chờ hoặc xử lý lỗi

Cách xử lý:

♦ a. Ghi log để phục hồi:

- Mỗi nút trước khi trả "OK" đều ghi log "**prepare**"
- Khi nút khởi động lại, đọc log để biết đang ở trạng thái nào:
 - Nếu log chỉ có "**prepare**" → có thể rollback an toàn
 - Nếu đã ghi "**commit**" → tiếp tục hoàn tất giao tác

♦ b. Coordinator phát hiện lỗi:

- Coordinator sử dụng **timeout** → nếu nút không phản hồi trong thời gian quy định:
 - Quyết định "**ABORT**" toàn bộ giao tác
 - Gửi "**ROLLBACK**" cho các nút khác

♦ c. Dự phòng và phục hồi:

- Dùng cơ chế **replica (dự phòng)** hoặc **failover**:
 - Khi nút chính lỗi, nút phụ tiếp nhận và hoàn tất giao tác
- Coordinator cần giữ **log giao tác toàn cục**, cho phép các nút khôi phục trạng thái đúng

Câu 3:

a. Trình bày ba cách tiếp cận chính trong điều khiển tương tranh: (1) bằng khóa (pessimistic), (2) lạc quan (optimistic), và (3) dựa trên nhãn thời gian (timestamp-based). Nêu đặc điểm cơ bản của mỗi phương pháp.

1. Điều khiển tương tranh bằng khóa (Pessimistic Concurrency Control)

Đặc điểm chính:

- **Quan điểm bi quan:** Giả định xung đột sẽ xảy ra nên chủ động ngăn chặn bằng cách sử dụng **cơ chế khóa (lock)**.
- Giao tác muốn truy cập dữ liệu phải **thiết lập khóa** tương ứng (đọc hoặc ghi). Chỉ khi giữ khóa thì mới được phép thao tác.
- Khi kết thúc giao tác (COMMIT/ROLLBACK), hệ thống **giải phóng các khóa**.
- Sử dụng **mô hình khóa hai pha (2PL)**:
 - **Pha 1:** thiết lập và nâng cấp khóa.
 - **Pha 2:** sau thao tác cuối, bắt đầu nhả khóa (thường hoặc nghiêm ngặt).
- **Đảm bảo tính tương đương tuần tự.**
- Có thể dẫn đến hiện tượng **khóa chết (deadlock)** nếu hai giao tác chờ nhau giải phóng khóa.

Giải pháp xử lý khóa chết:

- Khóa toàn bộ dữ liệu từ đầu (giảm hiệu năng).
- Khóa theo thứ tự định trước.
- **Dùng bộ giám sát phát hiện và hủy giao tác** có thời gian bảo vệ quá lâu để phá vỡ vòng chờ.

2. Điều khiển tương tranh lạc quan (Optimistic Concurrency Control)

Đặc điểm chính:

- **Quan điểm lạc quan:** Cho rằng xung đột hiếm khi xảy ra nên không cần khóa; chỉ xử lý xung đột khi nó thực sự xảy ra.
- Giao tác được chia làm **3 pha**:
 1. **Pha chuẩn bị:** Thực hiện trên các phiên bản tạm (bản sao gần nhất); kiểm tra ràng buộc.
 2. **Pha phê chuẩn:** Kiểm tra xung đột với các giao tác khác dựa trên **luật đọc/ghi** (Bảng 6.2).
 3. **Pha cập nhật:** Nếu phê chuẩn thành công, cập nhật bản ghi chính thức.

Phân loại kiểm tra phê chuẩn:

- **Phê chuẩn ngược** (backward validation): Kiểm tra tập **đọc** của giao tác đang chờ với tập **ghi** của các giao tác đã cam kết.
- **Phê chuẩn xuôi** (forward validation): Kiểm tra tập **ghi** của giao tác đang chờ với tập **đọc** của các giao tác đang hoạt động.

Nếu phát hiện xung đột, có thể:

- Hủy giao tác hiện tại.
- Hủy các giao tác khác.
- Hoặc **chờ và kiểm tra lại** sau.

3. Điều khiển tương tranh dựa trên nhãn thời gian (Timestamp-based Concurrency Control)

Đặc điểm chính:

- Mỗi giao tác được gán một **nhãn thời gian duy nhất** khi bắt đầu.
- Thứ tự thực hiện giao tác tuân theo **thứ tự nhãn thời gian** (giao tác có nhãn nhỏ hơn phải được xử lý trước).
- Dựa trên **luật thứ tự thao tác**:

- Giao tác chỉ ghi nếu các giao tác trước đã hoàn thành đọc/ghi.
- Giao tác chỉ đọc nếu các giao tác trước đã ghi xong.

Ưu điểm:

- **Không cần khóa**, tránh được **khóa chết**.
- Cho phép thực hiện song song trên các **phiên bản tạm thời**.

Quy trình xử lý gồm hai giai đoạn:

1. **Phê chuẩn:** Kiểm tra nhãn thời gian của các giao tác để đảm bảo trật tự thực hiện.
2. **Thực hiện:** Thực hiện thao tác hoặc hủy ngay nếu vi phạm thứ tự.

b. Giải thích khái niệm “tương đương tuần tự” (serializability): tại sao chỉ áp dụng luật xung đột đọc/ghi là chưa đủ, và làm thế nào để đảm bảo rằng một lịch (schedule) giao tác là tương đương tuần tự?

• Giải thích khái niệm “tương đương tuần tự” (serializability)

Tương đương tuần tự (serializability) là tính chất quan trọng trong điều khiển **tương tranh**, đảm bảo rằng kết quả thực hiện song song nhiều giao tác **không làm sai lệch tính đúng đắn** của hệ thống so với khi thực hiện tuần tự từng giao tác.

Một lịch (schedule) được gọi là **tương đương tuần tự** nếu **kết quả thực hiện của nó giống với một lịch tuần tự** nào đó (trong đó các giao tác được thực hiện toàn bộ theo một thứ tự nối tiếp, không xen kẽ).

• Tại sao chỉ áp dụng luật xung đột đọc/ghi là chưa đủ?

Luật xung đột đọc/ghi (xem **Bảng 6.1** trong dữ liệu):

- Đọc–Đọc: **Không xung đột**
- Đọc–Ghi / Ghi–Đọc / Ghi–Ghi: **Có xung đột**

Tuy nhiên, **chỉ xét xung đột từng cặp lệnh là chưa đủ**, vì:

- Có thể có **nhều giao tác và nhiều thao tác xen kẽ** nhau, gây ra **hiện tượng chờ lẫn nhau (deadlock)** hoặc **kết quả không đồng nhất**.
- Ví dụ: Hai giao tác T và U cùng cập nhật hai thực thể i và j nhưng theo **thứ tự khác nhau**, có thể dẫn tới **kết quả cuối không xác định hoặc sai lệch** nếu không đảm bảo thứ tự truy cập đồng nhất trên cả hai đối tượng.

⇒ Vì vậy, cần thêm một khái niệm mạnh hơn – đó là **tương đương tuần tự theo thứ tự truy cập dữ liệu**.

• Làm thế nào để đảm bảo một lịch là tương đương tuần tự?

Một lịch là tương đương tuần tự khi:

- Tất cả **các cặp thao tác có xung đột giữa các giao tác** đều thực hiện theo **cùng thứ tự trong lịch đó như trong một lịch tuần tự nào đó**.

Nói cách khác:

Nếu hai giao tác T và U cùng truy cập các đối tượng i và j, thì **T phải truy cập cả i và j trước U**, hoặc ngược lại. Không được để T truy cập i trước U, còn U lại truy cập j trước T.

Để đảm bảo điều đó, hệ thống sử dụng:

- **Cơ chế khóa tuần tự hóa (2PL)**: Không cho phép thao tác nếu chưa giữ được khóa.
- **Luật tương đương tuần tự theo thứ tự truy cập**: Như trong ví dụ của bạn, chỉ hợp lệ nếu **T thực hiện toàn bộ trên i và j trước U**, hoặc ngược lại.
- **Biểu đồ xung đột (conflict graph)**: Nếu không có chu trình trong đồ thị, thì lịch là tương đương tuần tự.

c. Cho hai giao tác T và U với các thao tác:

T: read(x); write(y) U: write(x); read(y)

a) Liệt kê tất cả các lịch khả dĩ (permutations) khi cho phép xen kẽ các thao tác.

b) Xác định lịch nào vi phạm tương đương tuần tự.

c) Theo phương pháp khóa hai pha nghiêm ngặt, hãy chỉ ra thứ tự cấp và nhả khóa với mỗi thao tác.

a) Liệt kê tất cả các lịch khả dĩ (permutations)

Tổng cộng có 4 thao tác:

- T1: read(x)
- T2: write(y)
- U1: write(x)
- U2: read(y)

Do các thao tác trong cùng một giao tác **phải giữ nguyên thứ tự**, nên ta cần liệt kê **mọi lịch xen kẽ (interleaved schedules)** mà vẫn đảm bảo:

- T1 trước T2
- U1 trước U2

Tất cả các lịch khả dĩ (đáp ứng ràng buộc trên):

1. T1 → T2 → U1 → U2
2. T1 → U1 → T2 → U2
3. T1 → U1 → U2 → T2
4. U1 → U2 → T1 → T2
5. U1 → T1 → U2 → T2
6. U1 → T1 → T2 → U2

→ Tổng cộng: 6 lịch hợp lệ

b) Xác định lịch nào vi phạm tương đương tuần tự

Để xác định vi phạm **tương đương tuần tự**, ta xét các **cặp thao tác xung đột**, tức là các thao tác trên cùng đối tượng dữ liệu và ít nhất một thao tác là ghi.

Các cặp thao tác có xung đột:

1. $T1: \text{read}(x) \times U1: \text{write}(x)$
2. $T2: \text{write}(y) \times U2: \text{read}(y)$

→ Để đảm bảo **tương đương tuần tự**, thứ tự các thao tác xung đột phải **giống nhau** trên toàn bộ lịch. Tức là:

- Nếu $T1 \rightarrow U1$, thì $T2$ phải đến trước $U2$
- Ngược lại, nếu $U1 \rightarrow T1$, thì $U2$ phải đến trước $T2$

Phân tích từng lịch:

Lịch	Thứ tự $T1/U1$	Thứ tự $T2/U2$	Có vi phạm?
1. $T1 \ T2 \ U1 \ U2$	$T1 \rightarrow U1$	$T2 \rightarrow U2$	Không
2. $T1 \ U1 \ T2 \ U2$	$T1 \rightarrow U1$	$T2 \rightarrow U2$	Không
3. $T1 \ U1 \ U2 \ T2$	$T1 \rightarrow U1$	$U2 \rightarrow T2$	Vi phạm
4. $U1 \ U2 \ T1 \ T2$	$U1 \rightarrow T1$	$U2 \rightarrow T2$	Không
5. $U1 \ T1 \ U2 \ T2$	$U1 \rightarrow T1$	$U2 \rightarrow T2$	Không
6. $U1 \ T1 \ T2 \ U2$	$U1 \rightarrow T1$	$T2 \rightarrow U2$	Vi phạm

c) Cấp và nhả khóa theo phương pháp khóa hai pha nghiêm ngặt (Strict 2PL)

Giả định:

- $read(x) \rightarrow$ cần khóa đọc $S(x)$
- $write(x) \rightarrow$ cần khóa ghi $X(x)$
- $read(y) \rightarrow$ cần khóa đọc $S(y)$
- $write(y) \rightarrow$ cần khóa ghi $X(y)$
- Theo Strict 2PL, tất cả khóa chỉ được nhả tại COMMIT/ROLLBACK

Thứ tự cấp và nhả khóa cho mỗi giao tác:

Giao tác T: $read(x); write(y)$

1. $read(x) \rightarrow$ cần $S(x)$
2. $write(y) \rightarrow$ cần $X(y)$
3. COMMIT $\rightarrow$ giải phóng $S(x), X(y)$

Giao tác U: $write(x); read(y)$

1. $write(x) \rightarrow$ cần $X(x)$
2. $read(y) \rightarrow$ cần $S(y)$
3. COMMIT $\rightarrow$ giải phóng $X(x), S(y)$

d. So sánh ưu nhược điểm của điều khiển tương tranh bằng khóa (pessimistic locking) và điều khiển lạc quan (optimistic concurrency control) trong kịch bản: một hệ thống chấm điểm trực tuyến (online grading) có rất nhiều người đọc (giáo viên, trợ giảng) và thỉnh thoảng có cập nhật (nhập điểm mới).

- o Phân tích tác động lên hiệu năng và nguy cơ khóa chết.
- o Xác định phương pháp nào phù hợp hơn và vì sao.

1. So sánh ưu – nhược điểm trong ngữ cảnh hệ thống chấm điểm

Tiêu chí	Khóa (Pessimistic Locking)	Lạc quan (Optimistic Concurrency Control)
----------	----------------------------	---

Cách tiếp cận	Giao tác giữ khóa khi đọc/ghi để loại trừ tương hỗ.	Cho phép tự do thao tác, kiểm tra xung đột tại thời điểm commit.
Hiệu quả khi đọc nhiều, ghi ít	Không tối ưu do vẫn cần thiết lập khóa, kể cả khi chỉ đọc.	Rất hiệu quả vì đọc không cần khóa, giảm chi phí đồng bộ.
Tính đồng thời	Thấp hơn do phải chờ khóa và có thể gây nghẽn tài nguyên.	Cao hơn vì cho phép thực hiện song song nhiều giao tác.
Nguy cơ khóa chết (deadlock)	Có – nếu các giao tác khóa chéo tài nguyên.	Không – vì không sử dụng khóa lâu dài.
Chi phí phục hồi nếu xung đột	Ít hơn vì xung đột bị ngăn chặn từ đầu.	Cao hơn do giao tác có thể bị huỷ sau khi đã thực hiện xong toàn bộ.
Độ trễ khi cập nhật	Có thể cao nếu giao tác bị chặn lâu do giữ khóa.	Thấp hơn nếu không có xung đột; nếu có, phải thực hiện lại.
Khả năng mở rộng hệ thống	Hạn chế hơn do cần quản lý nhiều khóa và tài nguyên dùng chung.	Tốt hơn vì giảm thiểu đồng bộ, phù hợp hệ thống nhiều truy cập đọc.

2. Phân tích tác động đến hiệu năng và nguy cơ khóa chết

Điều khiển bằng khóa:

- **Hiệu năng:**

- Giảm khi có nhiều người đọc vì ngay cả thao tác đọc cũng có thể bị chặn nếu có khóa ghi.
- Tạo độ trễ cho người dùng vì phải chờ giữ hoặc giải phóng khóa.

- **Khóa chết:**

- Nguy cơ cao nếu nhiều người thao tác ghi đồng thời (ví dụ: hai giảng viên nhập điểm cùng lúc).
- Cần cơ chế phát hiện & phục hồi (hủy giao tác, giới hạn thời gian khóa...).

Điều khiển lạc quan:

- **Hiệu năng:**
 - Tối ưu trong môi trường "nhiều đọc – ít ghi" vì không khóa dữ liệu khi đọc/ghi tạm thời.
 - Giao tác bị huỷ khi phát hiện xung đột ở pha phê chuẩn, nhưng điều này hiếm khi xảy ra nếu ghi ít.
- **Khóa chết:**
 - **Không xảy ra** do không sử dụng khóa lâu dài trên tài nguyên.

3. Phương pháp nào phù hợp hơn? Vì sao?

Phù hợp hơn: Điều khiển tương tranh lạc quan (Optimistic Concurrency Control)

Lý do:

- Hệ thống chấm điểm trực tuyến có **tỉ lệ đọc rất cao** (nhiều giáo viên, trợ giảng xem điểm).
- Việc **ghi/nhập điểm xảy ra thỉnh thoảng**, tức là **xác suất xung đột thấp** → phù hợp với giả định của điều khiển lạc quan.
- **Hiệu năng tốt hơn** khi phần lớn các giao tác là đọc – không cần khóa → giảm tắc nghẽn.
- **Tránh được khóa chết** – không cần cơ chế xử lý phức tạp như timeout hay phát hiện deadlock.
- Nếu có xung đột cập nhật, chỉ cần cho phép người dùng cập nhật lại (nhập lại điểm), điều này thường chấp nhận được trong hệ thống như chấm điểm.

e. Thiết kế một cơ chế điều khiển tương tranh kết hợp (hybrid) cho một hệ thống cơ sở dữ liệu phân tán hỗ trợ giao dịch ngân hàng:

- Đề xuất cách phân vùng dữ liệu và chọn phương pháp (khóa, lạc quan hay nhấn thời gian) cho từng phân vùng tùy tính chất giao dịch.
- Trình bày quy trình khi gặp xung đột (khi nào rollback, khi nào chờ).

- **Đánh giá ưu nhược điểm của giải pháp kết hợp này so với áp dụng thuần túy một trong ba phương pháp.**

1. Phân vùng dữ liệu và lựa chọn cơ chế điều khiển tương tranh

Trong hệ thống ngân hàng phân tán, có thể chia dữ liệu thành **các phân vùng** theo **tài khoản**, **chi nhánh**, hoặc **loại tác vụ**. Tùy theo tính chất tương tác và mức độ xung đột tiềm năng của mỗi phân vùng, ta chọn phương pháp điều khiển tương tranh phù hợp:

Phân vùng dữ liệu	Đặc điểm giao dịch	Cơ chế tương tranh được chọn
Tài khoản ngân hàng (tài khoản cá nhân)	Giao dịch ghi (rút, chuyển tiền)	Khóa hai pha nghiêm ngặt (2PL)
Bảng lãi suất, danh sách ngân hàng	Chủ yếu đọc, cập nhật hiếm	Lạc quan (OCC)
Lịch sử giao dịch	Ghi nối tiếp, không sửa	Nhãn thời gian hoặc OCC

Giải thích:

- **2PL nghiêm ngặt** dùng cho **tài khoản** vì cần bảo toàn **tính toàn vẹn** và **tuần tự hóa chặt** (tránh mất tiền, mất cập nhật).
- **OCC** phù hợp với bảng **thông tin ít thay đổi** như tỷ giá, vì xung đột ít xảy ra, giảm chi phí khóa.
- **Nhãn thời gian** cho các bảng ghi theo dòng thời gian như **lịch sử giao dịch**, cho phép nhiều giao tác đồng thời ghi mà vẫn bảo toàn trình tự.

2. Quy trình xử lý xung đột

Tùy theo cơ chế từng phân vùng, xử lý xung đột sẽ khác nhau:

a. Phân vùng dùng khóa (2PL nghiêm ngặt)

- Nếu đối tượng bị khóa bởi giao tác khác và không tương thích $\Rightarrow$ **Giao tác phải chờ**.
- Nếu xảy ra **khóa chết**, hệ thống dùng **giám sát thời gian**: phát hiện vòng chờ $\Rightarrow$ **Rollback** giao tác lâu nhất.

- Khi giao tác kết thúc (COMMIT hoặc ABORT) $\Rightarrow$ giải phóng khóa.

b. Phân vùng dùng OCC

- Trong **pha phê chuẩn**, nếu phát hiện:
 - **Tập đọc chồng với tập ghi giao tác đã cam kết trước đó $\Rightarrow$ Rollback giao tác đang phê chuẩn** (phê chuẩn ngược).
 - **Tập ghi chồng với tập đọc các giao tác đang chạy $\Rightarrow$ Rollback giao tác đang phê chuẩn hoặc các giao tác xung đột khác** (phê chuẩn xuôi).
- Có thể **đợi rồi kiểm tra lại** nếu giao tác quan trọng.

c. Phân vùng dùng nhãn thời gian

- Nếu yêu cầu ghi/truy xuất vi phạm trình tự nhãn thời gian $\Rightarrow$ **Giao tác bị hủy ngay (rollback)**.
- Trong một số trường hợp, có thể **chờ giao tác có nhãn nhỏ hơn hoàn thành** rồi tiếp tục (nếu hệ thống cho phép trì hoãn).

3. Đánh giá cơ chế điều khiển tương tranh kết hợp (hybrid)

Ưu điểm:

- **Tối ưu hiệu năng tổng thể:** Từng phân vùng được xử lý bởi cơ chế phù hợp nhất với đặc điểm giao dịch.
- **Giảm nguy cơ khóa chết:** Vì chỉ vài phân vùng sử dụng khóa nghiêm ngặt, phần còn lại dùng OCC/nhãn thời gian vốn không gây chờ đợi.
- **Tăng khả năng song song:** OCC và nhãn thời gian cho phép các giao tác đọc ghi đồng thời nhiều hơn.
- **Tăng tính mở rộng:** Hệ thống có thể dễ dàng bổ sung phân vùng mới với cơ chế tương ứng.

Nhược điểm:

- **Phức tạp khi cài đặt:** Cần thiết kế và đồng bộ nhiều chiến lược điều khiển khác nhau.
- **Chi phí xác định phân vùng phù hợp:** Phải có thống kê hoặc đo lường mức độ xung đột từng phân vùng.
- **Phát sinh lỗi giao thoa:** Nếu một giao dịch truy cập nhiều phân vùng với cơ chế khác nhau $\Rightarrow$ khó đảm bảo tuần tự hóa toàn cục.
- **Chi phí kiểm tra phê chuẩn OCC cao:** Với hệ thống nhiều giao tác đang hoạt động, việc kiểm tra xung đột có thể tốn thời gian.